

Exercises week 7

prof. Richard Torkar

October 11, 2025

Ce que nous connaissons est peu de chose, ce que nous ignorons est immense.

Pierre-Simon Laplace

EXERCISE 1. Now it's time to get our hands wet on a controversial dataset.¹ This dataset and the study that originally published it, stirred a lot of controversy (it's as close to all out war as it can get in academia...) ² We're gonna have a look at it and see if what the original authors claimed is, well, correct. They wanted to study what the effect of programming languages is on software quality. They claimed the design of a language had a significant effect on software quality.

First, we would need to download the dataset (yes, it's large!) and format it. However, we only need a small set of the data for the exercise, so I've just put it as a CSV online for you to download,

```
# where is the file?
url <- "https://torkar.github.io/tosem.csv"
# where should we place the file?
destFile <- "~/Downloads/tosem.csv"
# d/l file
download.file(url, destFile)
# read the file into d
d <- read.csv("~/Downloads/tosem.csv", sep = ";")
dim(d)
```

by executing the last call you can check that we have 1127 rows and 3 columns. The first column is `project`, which is a unique ID for each project (a project may have several rows in the data). Next, `n_bugs` is our outcome (the lower number of bugs, the better). Finally, `language_id` is what we are really interested in, i.e., the language used.

Now, create a number of multilevel models with language and/or project as varying intercepts. Compare all models with LOO. Finally, do we see a significant difference between the 17 languages? Can you infer something from the variability of projects and languages?

¹ B. Ray, D. Posnett, V. Filkov, and P. Devanbu. A large scale study of programming languages and code quality in GitHub. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 155–165, 2014. doi: 10.1145/2635868.2635922

² Read more about it here: <https://www.hillelwayne.com/post/this-is-how-science-happens/>

Be very careful when you copy code!

Use hyperparameters/hyperpriors.

SOLUTION 1. First, we should consider the ontological and epistemological justifications concerning the underlying data generation process. Our outcome is a count $0 \rightarrow \infty$ (recognize this?) The information theoretical justification (epistemological) would be to rely on the maximum entropy distribution for counts, i.e., low probability and many trials, which is the Poisson. But, look at the mean ($\bar{x} \approx 500$) and the variance ($s^2 = 15031006$) in our outcome. It's obvious we will face challenges down the road. I wish one would at least once get data that can be used for fitting a Poisson model. . .

No need to whine about it; instead we rely on the Negative-Binomial, which can model the variance separately. Let's create a set of models $M = M_0, \dots, M_n$ and compare them with LOO. Please, look carefully at each model specification below and try to figure out what I tried to accomplish. Also, the priors might seem tight, but they are actually very broad (please do prior predictive checks and check for yourself)—remember, we use a log link and that makes funky things to the priors.

After the models, further down below, I explain the main differences between the models, and what I wanted to achieve. The lines you should pay special attention to end with a hash (#) and a number.

```
set.seed(061215)
m0 <- ulam(
  alist(
    n_bugs ~ dgampois(lambda, phi),
    log(lambda) <- a_l[language_id],
    a_l[language_id] ~ dnorm(a_bar, sigma), #1
    a_bar ~ dnorm(0, 1),
    sigma ~ dexp(1),
    phi ~ dexp(1)
  ), data = d, cores = 4, chains = 4, cmdstan = TRUE,
  log_lik = TRUE, iter = 5e3
)

m1 <- ulam(
  alist(
    n_bugs ~ dgampois(lambda, phi),
    log(lambda) <- a_p[project],
    a_p[project] ~ dnorm(a_bar, sigma), #2
    a_bar ~ dnorm(0, 0.2),
    sigma ~ dexp(1),
    phi ~ dexp(1)
  ), data = d, cores = 4, chains = 4, cmdstan = TRUE,
  log_lik = TRUE, iter = 5e3
)

m2 <- ulam(
  alist(
    n_bugs ~ dgampois(lambda, phi),
    log(lambda) <- a_l[language_id] + a_p[project],
    a_p[project] ~ dnorm(0, sigma_p), #3
    sigma_p ~ dexp(1),
    a_l[language_id] ~ dnorm(0, sigma_l), #4
    sigma_l ~ dexp(1),
    phi ~ dexp(1)
  ), data = d, cores = 4, chains = 4, cmdstan = TRUE,
  log_lik = TRUE, iter = 5e3
)
```

```

)

m3 <- ulam(
  alist(
    n_bugs ~ dgamma(lambda, phi),
    log(lambda) <- a_l[language_id] + a_p[project],
    a_p[project] ~ dnorm(0, sigma_p),
    sigma_p ~ dexp(1),
    a_l[language_id] ~ dnorm(a_bar, sigma_l), #5
    a_bar ~ dnorm(0, 0.5),
    sigma_l ~ dexp(1),
    phi ~ dexp(1)
  ), data = d, cores = 4, chains = 4, cmdstan = TRUE,
  log_lik = TRUE, iter = 5e3
)

m4 <- ulam(
  alist(
    n_bugs ~ dgamma(lambda, phi),
    log(lambda) <- a_l[language_id] + a_p[project],
    a_p[project] ~ dnorm(a_bar_p, sigma_p), #6
    sigma_p ~ dexp(1),
    a_l[language_id] ~ dnorm(a_bar_l, sigma_l), #7
    a_bar_l ~ dnorm(0, 0.5),
    a_bar_p ~ dnorm(0, 0.2),
    sigma_l ~ dexp(1),
    phi ~ dexp(1)
  ), data = d, cores = 4, chains = 4, cmdstan = TRUE,
  log_lik = TRUE, iter = 1e4
)

```

So, $\mathcal{M}_0$ is a multi-level model where we model language as a varying intercept. If you look at the hash sign you see that we declare a prior, which then consists of other parameters, e.g., a_bar is a hyperparameter and we set a hyperprior on the next line. $\mathcal{M}_1$ is the same but instead we model projects.

In $\mathcal{M}_2$ we then use both project and language. Here, however, we model the variance, σ_l and σ_p , separately. In short, I have reason to believe that modeling this carefully will tell us something. For $\mathcal{M}_3$ we model the mean of language (using hyperparameters), while in $\mathcal{M}_4$ we model the mean of both language and project (with hyperparameters).

Check the chains, $\widehat{R}$, and effective sample size. Only in $\mathcal{M}_4$ do we have reasons to worry. One of the parameters has $\widehat{R} = 1.01$, which is worrisome, but we still have effective sample size in the hundreds so I'll let it slide for now (you know what to do if you have $\widehat{R} > 1.01$!)

Our models now estimate hundreds of parameters. Let's have a look at how many effective parameters we're estimating now,

	PSIS	SE	dPSIS	dSE	pPSIS	weight
m3	14149.5	128.77	0.0	NA	316.9	0.62
m2	14150.5	128.26	1.0	4.20	316.1	0.37
m4	14159.0	129.44	9.5	4.25	321.5	0.01
m1	14410.0	138.18	260.5	37.17	367.5	0.00
m0	14580.0	154.15	430.5	68.70	53.0	0.00

so the winning model has ≈ 317 effective parameters, but in reality we model

Varying intercepts are also called varying effects, random effects, or group-level effects, I can't say that any of those names are particularly good, but probably exist only to confuse people.

	mean	sd	5.5%	94.5%	n_eff	Rhat4
sigma_p	1.08	0.06	0.99	1.17	3805	1.00
a_bar_l	1.40	0.62	0.44	2.42	7379	1.00
a_bar_p	0.22	0.20	-0.11	0.54	351	1.01
sigma_l	3.37	0.81	2.16	4.74	2531	1.00
phi	0.74	0.03	0.68	0.79	7874	1.00

751 parameters. Thanks to our regularizing hyperpriors we avoid overfitting!

Also, you will see some warnings from LOO about high Pareto k values, but rerunning with pointwise comparison will lead to the same output.

In the output, we clearly see that the models are divided into two clusters. One in which they have a lower PSIS ($\mathcal{M}_3, \mathcal{M}_2, \mathcal{M}_4$) and one that seems to not catch up, i.e., $\mathcal{M}_1$ and $\mathcal{M}_0$. What LOO is saying is that having both project and language might be a good idea (i.e., not models $\mathcal{M}_0$ and $\mathcal{M}_1$). Additionally, $\mathcal{M}_3$ and $\mathcal{M}_2$ are significantly better than $\mathcal{M}_4$.³ What does this mean?

$$^3 9.5 + c(-1,1) * 4.25 * 1.96 = [1.17, 17.83]$$

Well, after having accounted for modeling the variance using hyperparameters for both language and project ($\mathcal{M}_2$), focusing on modeling the mean for language using hyperparameters ($\mathcal{M}_3$) does not give much more. Also, modeling mean and variance for both language and project using hyperparameters ($\mathcal{M}_4$), does not do that much either.

Let us have a look at the standard deviation using `precis(m3)`,

	mean	sd	5.5%	94.5%	n_eff	Rhat4
sigma_p	1.08	0.06	0.99	1.17	2569	1
a_bar	1.29	0.59	0.36	2.24	12383	1
sigma_l	3.65	0.77	2.55	4.98	11899	1
phi	0.74	0.03	0.68	0.79	6720	1

so the standard deviation for project is lower (1.08) than for language (3.65). Let us plot a random selection of projects and all our 17 languages (next page),

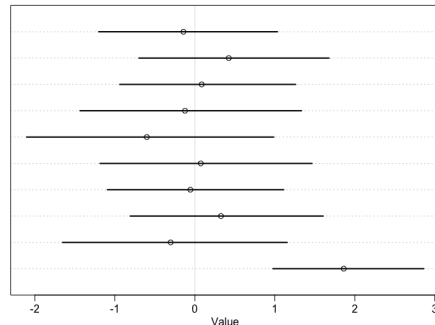


Figure 1: Project estimates

and we see that there is clearly more variability among languages. Although we do have variability among projects, which we should take into account.

If we examine the language estimates, we see that some languages have a lower number of bugs than others. So, what is the conclusion? Should we use Languages 12 and 17 in projects from now on? Well, perhaps...

But first we need to consider omitted variable bias; we have more variables in the data that we could take into account (which you have not downloaded). When we do that, we risk adding colliders and whatnot. Also, this analysis does not say much in itself—we account for variance from two sources, project and language, but that is it.

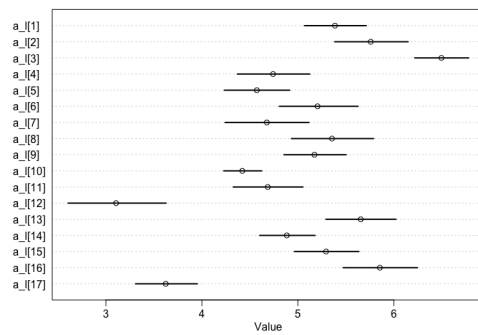


Figure 2: Language estimates

If you are curious, according to me and my coauthors, the answer to the question of which language to use is: depends.⁴

⁴ C. A. Furia, R. Torkar, and R. Feldt. Applying Bayesian analysis guidelines to empirical software engineering data: The case of programming languages and code quality. *ACM Transactions on Software Engineering and Methodology*, 31(3), mar 2022. ISSN 1049-331X. URL <https://dl.acm.org/doi/pdf/10.1145/3490953>